Development of an interactive web app for visualizing cancer evolution

Marie Dogo[1,2] and Luo Xiang Ge[1]

[1]Department of Biosystems Science and Engineering, ETH Zurich
[2]Ecole Nationale Supérieure des Mines de Paris, PSL

Abstract

Motivation: This work presents an interactive framework for visualizing and exploring tumor clonal evolution by combining mutation trees with fishplot-based temporal representations. A major strength of this approach lies in its ability to bridge the gap between phylogenetic reconstruction and temporal visualization. While existing methods such as SCITE or COMPASS accurately infer mutation trees, they often lack intuitive tools to explore how clonal populations evolve over time under different conditions.

Results: The proposed application addresses this limitation by coupling tree structures with dynamic abundance profiles, enabling users to better interpret tumor heterogeneity and its progression. Another important contribution is the incorporation of biological constraints directly into the editing process. By enforcing rules such as parent-child consistency, total abundance limits, and root normalization, the framework ensures that user-defined modifications remain biologically plausible. This is particularly relevant in an interactive setting, where unconstrained manual edits could otherwise lead to unrealistic clonal configurations. The introduction of drug effect simulation further enhances the exploratory capabilities of the tool. The multiplicative model provides a simple yet interpretable way to represent treatment impact on clonal populations.

Conclusion: this application offers a novel and intuitive approach to exploring tumor evolution, combining visualization, interactivity, and biological constraints. It represents a valuable tool for research visualization, while also highlighting opportunities for further methodological developments.

Keywords: Cancer genomics, Visualization, Tumor evolution, Fishplots, Web Engineering, Mutation Trees

Contents

1. Introduction

1.1 The tumor evolutionary trajectory

The progression of a tumor follows a specific evolutionary trajectory, starting from a single mutation in a founder cell and giving rise to multiple subclones that accumulate additional mutations during proliferation. This process underlies the phenomenon of cancer heterogeneity[9]. The initial mutation confers a selective advantage to the founder cell, enabling it to escape the regulatory mechanisms that maintain tissue homeostasis and can then proliferate uncontrollably. As cell divisions continue, the tumor genome evolves, leading to increasing genetic and phenotypic diversity compared to the original tumor. Intratumoral heterogeneity is a major cause of relapse[7][8] as a therapy can be effective against the dominant subclone at a given time and may be ineffective against other subclones, which can then expand. Moreover, it has been demonstrated that the order[1]in which mutations were acquired influenced clinical features along with the response to targeted therapy. Therefore, characterizing this heterogeneity and more importantly, to make reliable predictions of future evolutionary steps, are essential for predicting treatment responses and designing treatment strategies[10] that target the different subclonal populations at appropriate intervals to maximize therapeutic efficacy.
However, tracking the history evolution of tumors faces many challenges. Indeed, the diagnosis of a tumor often happens at a certain time when the primitive cell already underwent many mutations, as for lung cancers, where most diagnoses are set at advanced stage. In addition, reconstructing the past of the tumors is challenging for solid tumors, where sample are only available throught biospis, when the tumors are extracted.

1.2 Related works

Different methods have emerged for inferring the evolutionary history of tumours from scDNAseq data or bulk DNA sequencing samples[11]. Among the phylogenetic tree reconstruction methods from single-cell data we can cite SCITE[2] and COMPASS[3]. They enable to characterize which mutations happened in which order within a specific tumor, the mutation frequencies, how different subclones are related and how the tumor diversified over time. All these features are crucial to then describe the cancer progression using cancer progression models as FiTree[4], TreeMHN[5] or Oncotree2vec[12] that will learn the general rules or patterns that govern mutation order across the whole population.
The mutation tree, also referred to as a phylogenetic tree, provides a simple and effective way to visualize evolutionary history using either single-cell or bulk sequencing data, as shown in Fig. 1. Each node represents a distinct genetic event, and the edges connect these events to one another. Tumor cells are then mapped onto the corresponding mutation nodes to illustrate the evolutionary trajectory.

Once the trees have been reconstructed, their visualization can provide valuable insights into tumor heterogeneity and help in designing optimal therapeutic strategies. Another possibility is to

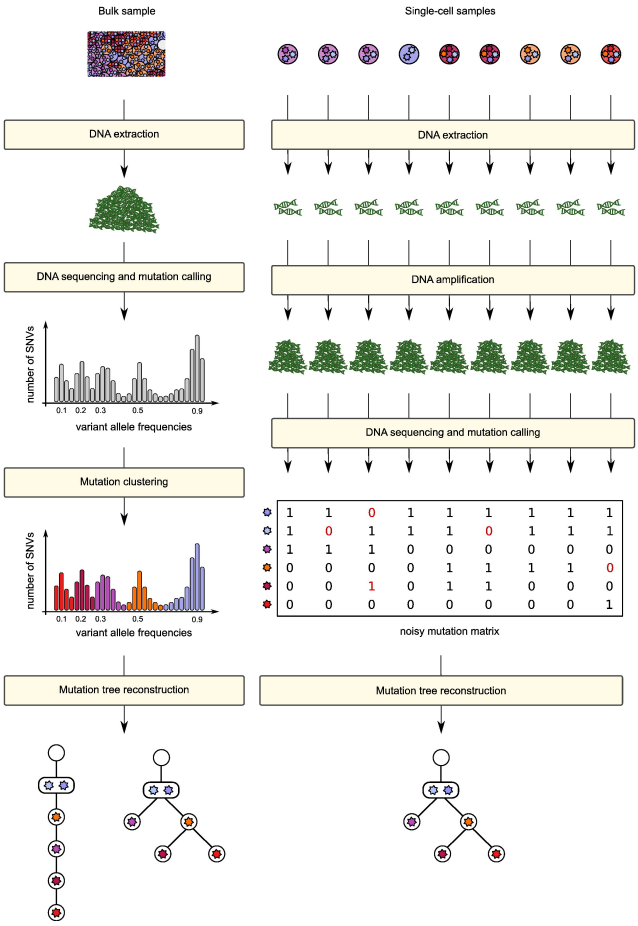


Figure 1: The mutation tree inference tools process the genomic sequencing data and reconstruct its phylogeny by a probabilistic approach using as taxa either the individual cells for single-cell data or the clones inferred based on variant allele frequencies, for bulk data. *Figure from Jack Kuipers and Katharina Jahn work* [11]

perform statistical analyses across different trees to identify conserved evolutionary trajectories. For instance, OncotreeVIS[12] is an interactive, web-based visualization tool designed to explore mutation trees across large tumor cohorts, facilitating the assessment of tumor heterogeneity. It allows users to view multiple trees simultaneously, organize them based on clustering results, or project them into a two-dimensional latent space derived from pairwise tree distances. The platform also integrates additional contextual data such as clinical information, drug–gene associations, recurrent mutation patterns, conserved evolutionary trajectories among tree groups, and k-nearest neighbor relationships.

1.3 Aim of this work

As previously described, many tools can accurately infer phylogenies; however, they often fail to provide intuitive and integrated visualizations of clonal evolution over time. This work addresses a key gap in existing methods by developing an interactive web application for visualizing tumor evolution, combining mutation trees with fishplot-based temporal dynamics.

The application also supports exploratory analysis of treatment effects through mutation-

specific drug impact simulations, providing immediate visual feedback. Importantly, biological consistency is maintained through built-in constraints (such as parent–child hierarchies and clone continuity), ensuring that the resulting visualizations are both interpretable and robust for research and clinical discussions.

Finally, the interactivity of the application enhances usability by allowing users to adjust key parameters, such as mutation proportions and the impact of drugs on individual mutations. It enables easy navigation across patients and timepoints, with automatic updates of the visualizations. Users can also add or remove timepoints, mutations, or drugs, and switch between different visualization formats to facilitate interpretation.

2. Usage and Implementation

2.1 Shiny App

The website is based on Shiny applications, as Shiny enables the creation of interactive, highly customizable web-based data visualizations and dashboards directly from R code. The fishplot library created by Chris Miller has been used to plot the fishplot[13]. The main challenge was adapting this package to my dataset, which differs substantially from the format expected by the original implementation, and many other constraints have been added in order to respect the biological reality. The remainder of this report details the work carried out on this library.

2.2 Lauching the app

The source code is publicly available on GitHub at: `https://github.com/marie-do`. After downloading the project folder from GitHub, the application can be launched in `RStudio` by running `app.R` that grouped all the code. Moreover, Using the correct working directory is required and if the working directory is not the `App` folder, these files may not be found and startup will fail.

Once `app.R` is executed, the Shiny application is initialized automatically. The code loads required packages and helper functions, then starts the interactive web interface. After a dataset is imported, the application builds clonal objects, computes hierarchical and temporal matrices, and updates visual mutation tree and fishplot dynamically according to user interactions.

Because the *fishplot* library cannot be installed online in every environment, the code implements a two-step strategy to handle the `fishplot` dependency. At startup, it first checks whether `fishplot` is already available in the local R environment. If the package is not installed, the application attempts an automatic installation from GitHub. This requires running the following commands in R, which are already implemented in my code:

```
install.packages("devtools")
library(devtools)
install_github("chrisamiller/fishplot")
```

If the installation succeeds, the official package is loaded directly and no further action is required. If it fails, the project falls back to a manual setup bundled within the repository, which

includes a local copy of the `fishplot` source code.

2.3 The Data

The development of the customized `fishplot` function is based on the Morita et al. (2020) dataset [14]. This dataset includes 153 acute myeloid leukemia (AML) point mutation trees from 123 patients, obtained using scDNA-seq. Tree inference was performed using the SCITE method [2]. The data are organized as a nested JSON-like structure representing clonal evolution trees with associated metadata. Each sample is named using the cancer type `"AML"`, followed by the sample identifier and a time-point qualifier. Each tree is composed of nodes linked hierarchically, where each node may contain one or more child nodes (`children`), except for leaves and each node includes several attributes as shown in table 1.

Table 1: Description of node attributes in the AML dataset

Attribute	**Type / Requirement**	**Description**
`node_id`	string / integer, required	Unique identifier for each node within the phylogenetic tree.
`label`	list, optional	Describes the node status; typically `["Root"]` for the ancestral clone, or an empty list for other nodes.
`name`	string	String representation of the node identifier.
`matching_label`	integer	Numerical identifier used to map nodes across different data representations (e.g., between the phylogenetic tree and mutation matrices).
`size_percent`	float, optional	Proportion of cells corresponding to the clone.
`gene_events`	dictionary, optional	Mapping of genes to somatic mutation annotations. Entries typically follow HGVS nomenclature (e.g., `NPM1: p.L287fs`, `DNMT3A: p.R882H`).

The reconstruction of the fishplot does not rely on all fields of the original dataset, but only on a subset of key attributes, namely `node_id`, `parent_id`, `size_percent`, `sample_id`, and the `mutation` variable derived from `gene_events`.
The dataset also includes a `metadata` field, which contains patient-level information such as `VitalStatus` (e.g., `"Alive NOS"` or `"Dead NOS"`). The overall structure follows a hierarchical JSON format similar to those used in visualization libraries such as `D3.js` or Python-based tree structures. An example is provided below.

Listing 1: Example of AML-63-005 sample tree in JSON format

```
"AML-63-005": {
```

```
  "tree": {
    "node_id": "4",
    "label": ["Root"],
    "variant": "",
    "name": "4",
    "matching_label": 4,
    "children": [
      {
        "node_id": "1",
        "size_percent": 0.511,
        "matching_label": 13,
        "gene_events": {
          "IDH2": {"SNV": "p.R140Q"}
        },
        "children": [
          {
            "node_id": "3",
            "size_percent": 0.489,
            "matching_label": 2,
            "gene_events": {
              "NPM1": {"SNV": "p.L287fs"}
            }
          }
        ]
      }
    ]
  },
  "metadata": {
    "VitalStatus": "Alive NOS"
  }
}
```


2.4 Interface presentation

Within the user interface, several interactive green control buttons, arranged horizontally, allow users to explore different visualization modalities as shown in Fig. 2.

- **FISHPLOT**: Displays the clonal evolution over time using a fishplot representation.
- **MUTATION TREE**: Visualizes the phylogenetic tree structure of mutations.
- **DATA INFO**: Provides detailed information about the currently loaded dataset (node attributes, structure, etc.).
- **DATA MATRIX**: Displays the mutation matrix representation of the data.

The vertical menu bar allows users to select a patient and corresponding timepoint, as well as choose a drug to visualize its impact on clonal evolution. It also provides the ability to edit

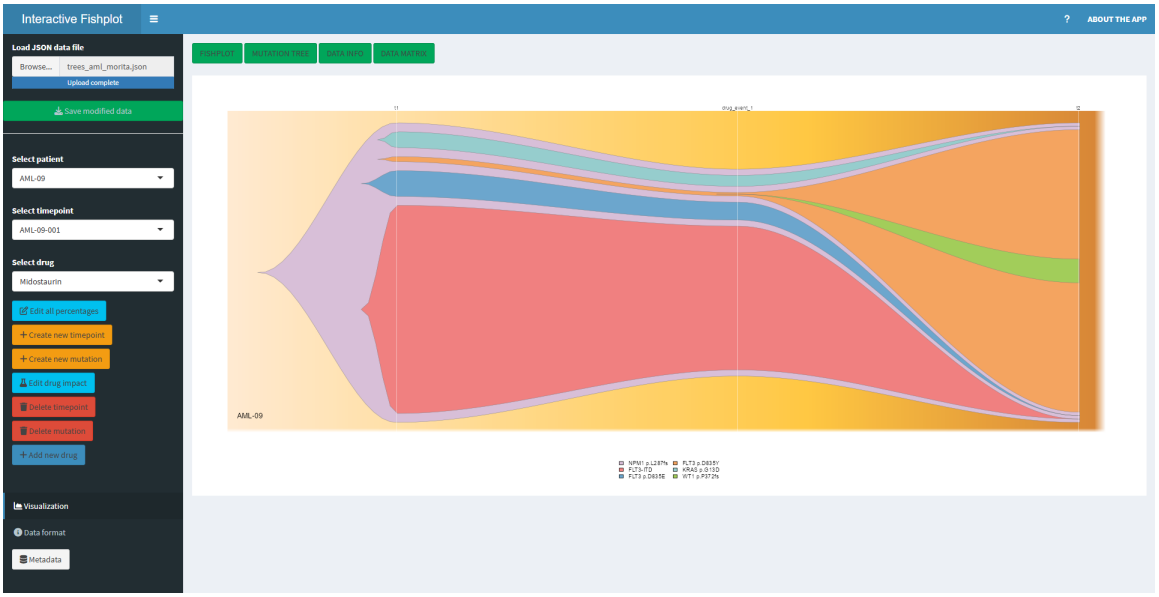


Figure 2: Interactive fishplot interface, featuring horizontally arranged green visualization buttons and vertically aligned controls for modifying the outputs.

mutation proportions individually and to add or remove mutations, timepoints, or drugs. A "Data Format" button summarizes the structure required by the application. Finally, in the top-right corner, a "Help" button assists users in navigating the app, while an "About the App" button directs them to the GitHub repository.

3. Features and Functionality

3.1 Samples Navigation

When the dataset is loaded, users can navigate through all samples using the *Select patient* control. If multiple timepoints are available, a specific timepoint can be selected via the *Select timepoint* option, allowing each timepoint to be visualized individually using the mutation tree representation.

For multi-timepoint samples, the fishplot includes at least two timepoints, each separated by drug events. These events represent the impact of a treatment over a given period. In contrast, mono-timepoint samples are displayed with two timepoints: the original timepoint t_1 and a duplicated version of t_1, since constructing a fishplot requires at least two timepoints.

The distinction between multi-timepoint and mono-timepoint fishplots is illustrated in Fig. 3.

3.2 Editing clones percentages

Clone proportions can then be adjusted through numeric input fields using the *Edit all percentages* button, with values constrained between 0 and 100. Each clone can be identified by interacting with the mutation tree as shown in Fig. 4, where hovering over elements reveals the corresponding mutation labels. To ensure biological consistency, several constraints are automatically enforced during editing. In particular, the proportion of a parent clone must

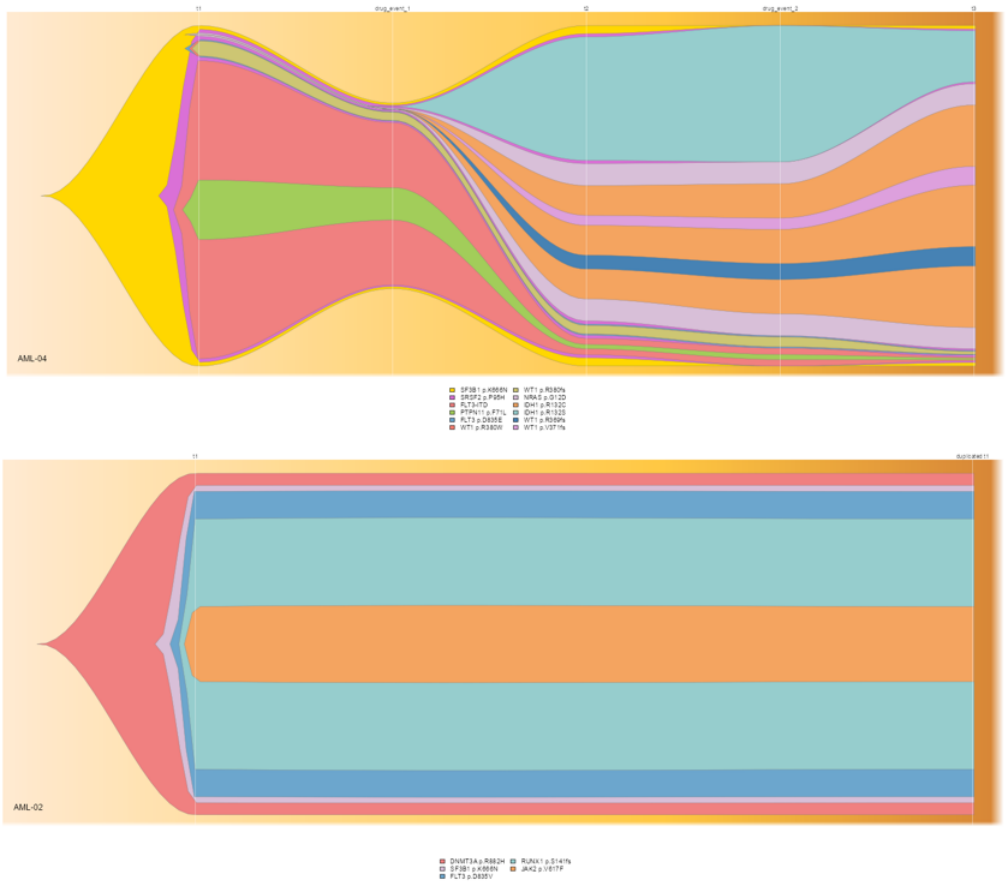


Figure 3: The first fishplot corresponds to a multi-timepoint sample with three timepoints, each separated by a drug event. The second fishplot represents a single-timepoint sample, in which the initial timepoint has been duplicated.

be greater than or equal to the sum of its descendant clones. If inconsistencies arise, the system applies automatic rebalancing to restore a valid hierarchy. Additional safeguards are implemented to maintain coherent clonal structures: the total proportion across all clones cannot exceed 100% and normalization is applied if it is the case. Moreover, the root clone remains fixed at 100%, and corrections are applied to prevent biologically implausible behaviors such as clone reappearance after extinction. Once modifications are complete, changes can be validated and immediately reflected in the updated visualization.
Here are the 3 major constraints, implemented in the code:

- **Total constraint:**

$$\sum_{i=1}^{N} x_i(t) \leq 100$$

- **Hierarchy constraint:** if j is a child of i, then:

$$x_j(t) \leq x_i(t)$$

- **Root constraint:**

$$x_{\text{root}}(t) = 100$$

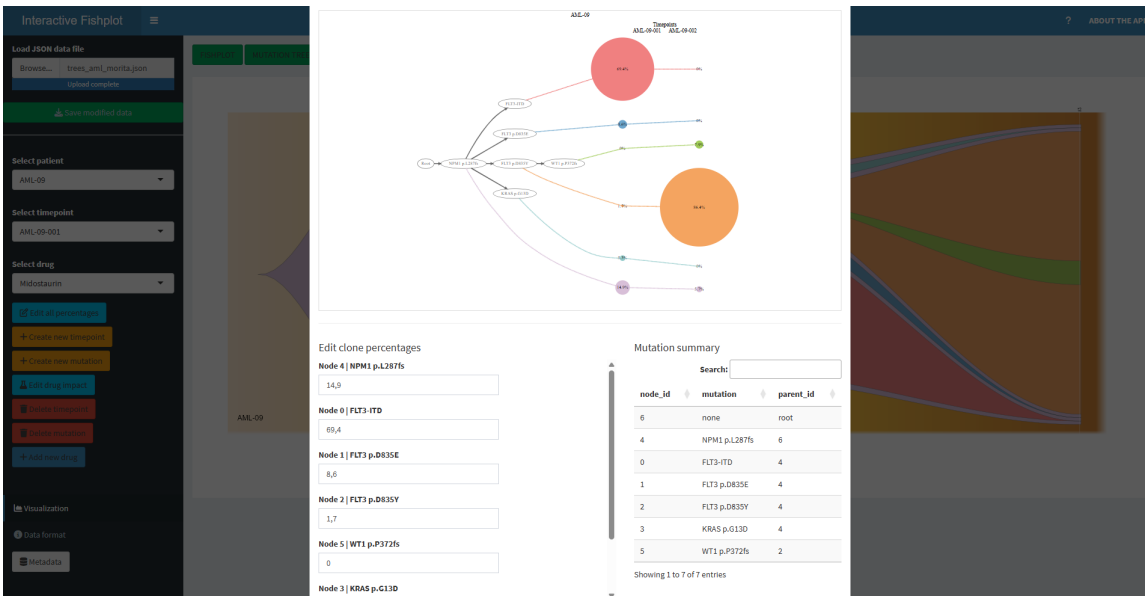


Figure 4: At the top corner, the mutation tree enables users to visualize the clonal hierarchy. In the bottom-right corner, a table summarizes the correspondence between node IDs and mutations, allowing users to identify each clone. This facilitates the modification of clone proportions in the bottom-left panel, where users can assign the desired values to each mutation.

3.3 Adding / Removing Timepoints

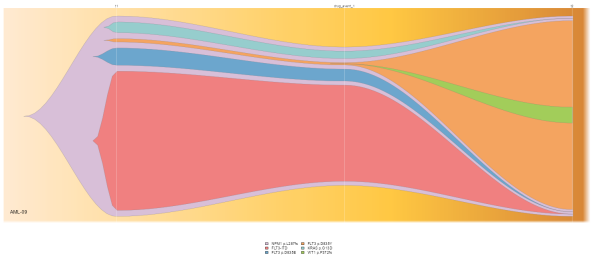
Users can add a new timepoint to a sample by selecting the *Add new timepoint* button in the left-side menu. This action automatically generates a new timepoint in which all mutation proportions are initialized to 0%. These values can subsequently be adjusted using the *Edit all percentages* option. The figure 5 illustrates the result of adding a timepoint to the AML-09 patient.

To remove a timepoint, the user must be positioned on the last timepoint, as deleting an intermediate timepoint would disrupt the temporal consistency of the data. The deletion must then be confirmed before being applied.

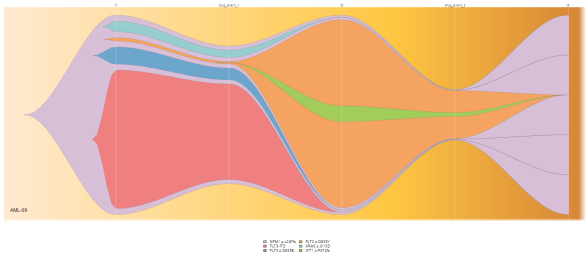
3.4 Adding/Removing mutations

3.4.1 Adding mutation

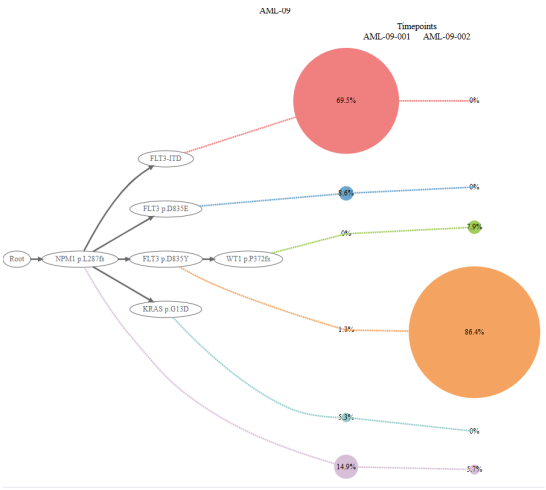
The application allows users to introduce new mutations within the clonal hierarchy at a selected timepoint. Figure 6 shows the interface the user must go through. Before creating a mutation, the desired timepoint must be selected using the corresponding selector in the interface. To define a new mutation, users first specify a parent clone, indicating the lineage from which the mutation arises. A mutation label can then be assigned, and an initial proportion is set for the selected timepoint, representing the size of the newly created clone. If the sum of all percentages exceeds



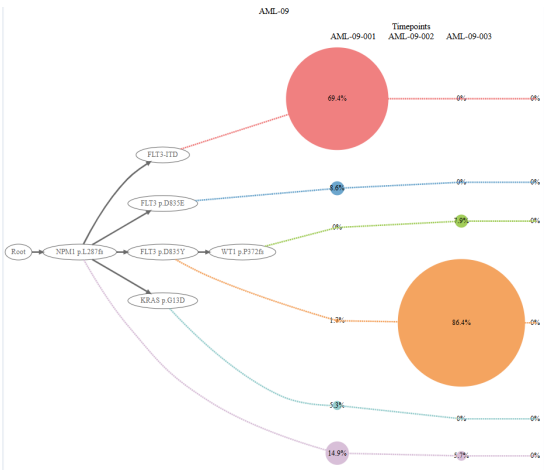
(a) Original Fishplot



(b) Updated Fishplot



(c) Original Mutation Tree



(d) Updated Mutation Tree

Figure 5: Adding a new timepoint for patient AML-09

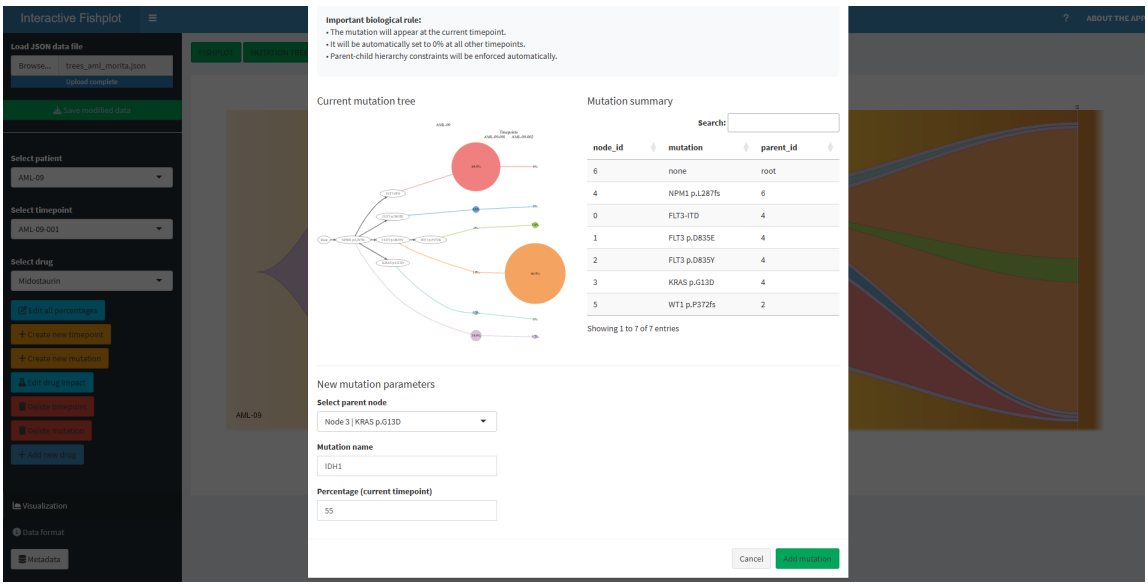
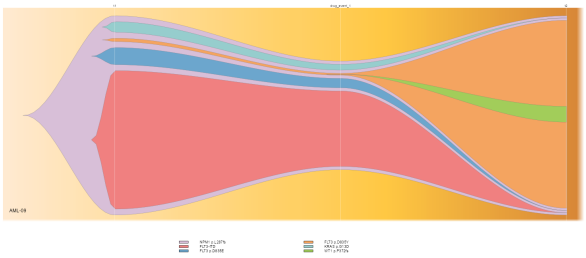
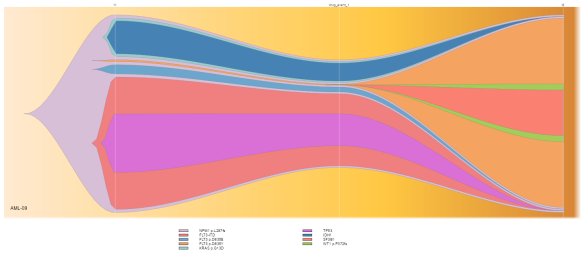


Figure 6: The *Add New Mutations* interface allows users to add as many mutations as they want. The mutation displayed at the top helps users better visualize the mutation hierarchy. In the bottom-right corner, users can associate each mutation with its specific node ID. They can then choose the parent of the new mutation, thereby defining its position in the updated mutation tree and fishplot. Finally, users must specify the name and the percentage size of the new mutation.

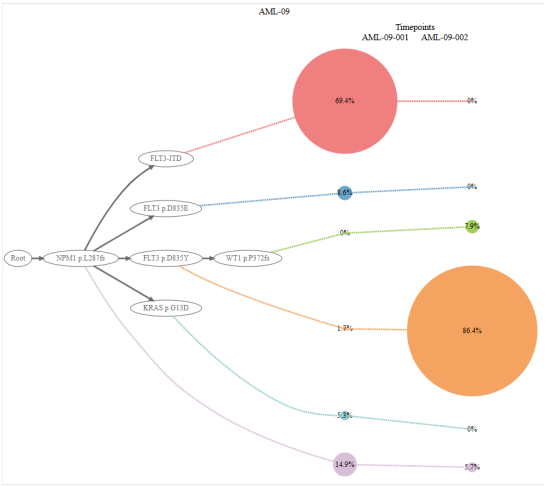
or falls below 100%, the values are automatically adjusted to maintain consistency. The system enforces biological constraints throughout this process. The new mutation is introduced only at the selected timepoint and is initialized to 0% at all other timepoints. In addition, parent–child hierarchy constraints are applied to ensure that the resulting clonal structure remains valid. Figure 8 illustrates the result of adding multiple mutations to the AML-09 patient.



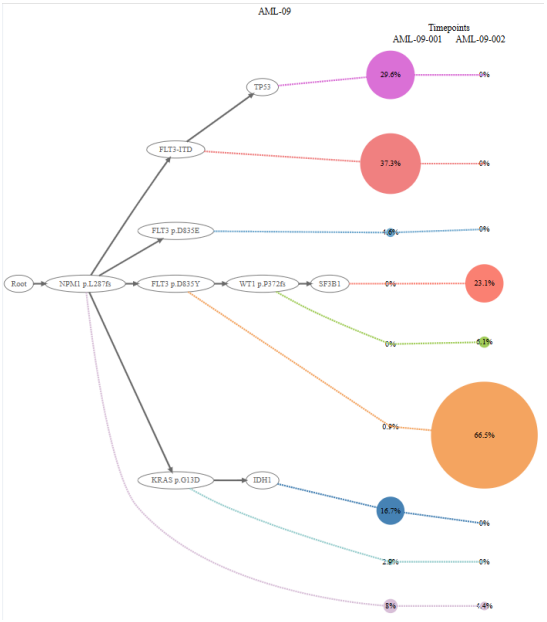
(a) Original Fishplot



(b) Updated Fishplot



(c) Original Mutation Tree



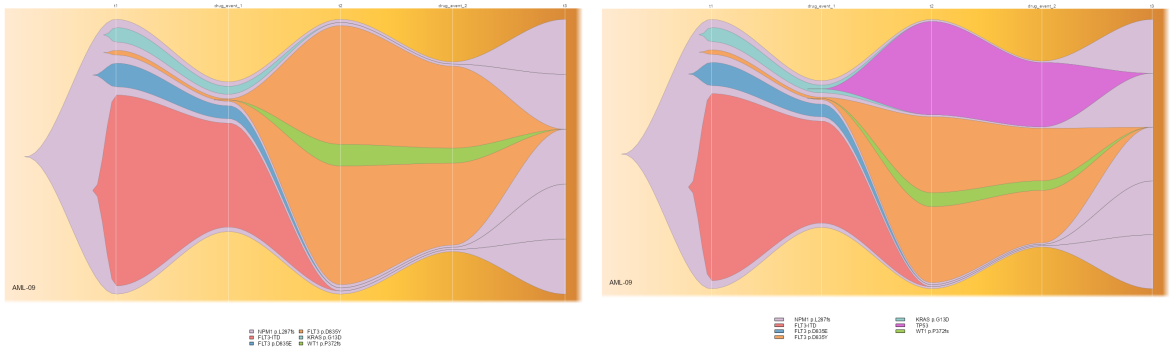
(d) Updated Mutation Tree

Figure 7: Addition of new mutations for patient AML-09. The IDH1, TP53, and SF3B1 mutations were added to the original patient data.

3.4.2 Automatic correction of zero-valued parents during mutation creation.

To ensure robust visualization of newly added clones, an automatic correction step is incorporated into the mutation–insertion workflow. When a user introduces a new mutation at time point t^* with parent clone p, the application first evaluates the parent abundance $x_p(t^*)$. If the parent is absent at t^* or has zero abundance, its value is automatically replaced with a small positive constant:

$$x_p(t^*) \leftarrow \varepsilon, \qquad \varepsilon = 10^{-6}.$$



(a) Original Fishplot (b) Fish plot after parent adjustment

Figure 8: Automatic correction of zero-valued parents during mutation creation, for AML-09 at timepoint 2. The parent is KRAS p.G13D and its new child (new created mutation) is TP53 initially set to 53% before normalization.

This adjustment is performed prior to inserting the child mutation, thereby preserving the validity of the parent–child lineage for Fishplot rendering without requiring manual intervention. The new mutation is then assigned a user-defined abundance at the selected time point and initialized to zero at all other time points:

$$x_{\text{new}}(t) = \begin{cases} x_{\text{new}}(t^*) & \text{if } t = t^*, \\ 0 & \text{otherwise.} \end{cases}$$

Following insertion, abundances are normalized and hierarchical constraints are enforced, ensuring biologically consistent and visually stable clonal trajectories.

3.4.3 Removing mutations

The application provides a controlled mechanism for removing mutations from the clonal hierarchy at a selected timepoint as shown in figure 9 . Prior to deletion, users must ensure that the appropriate timepoint is selected using the interface controls. To preserve biological and structural consistency, several constraints are enforced. Only leaf mutations, corresponding to terminal clones without descendants, can be removed. In contrast, root and internal nodes are protected to maintain the integrity of the clonal architecture.Additionally, mutations that are present across multiple timepoints cannot be entirely deleted, ensuring consistency with the evolutionary history. However, newly introduced mutations that appear only at the current timepoint can be fully removed.

3.5 Drugs effects

For each sample, a drug effect is simulated. This effect is introduced between consecutive real timepoints and aims to reproduce the evolution of each individual mutation under treatment. The user can select a drug from the left-side menu. Currently, four drugs are available: *Midostaurin*,

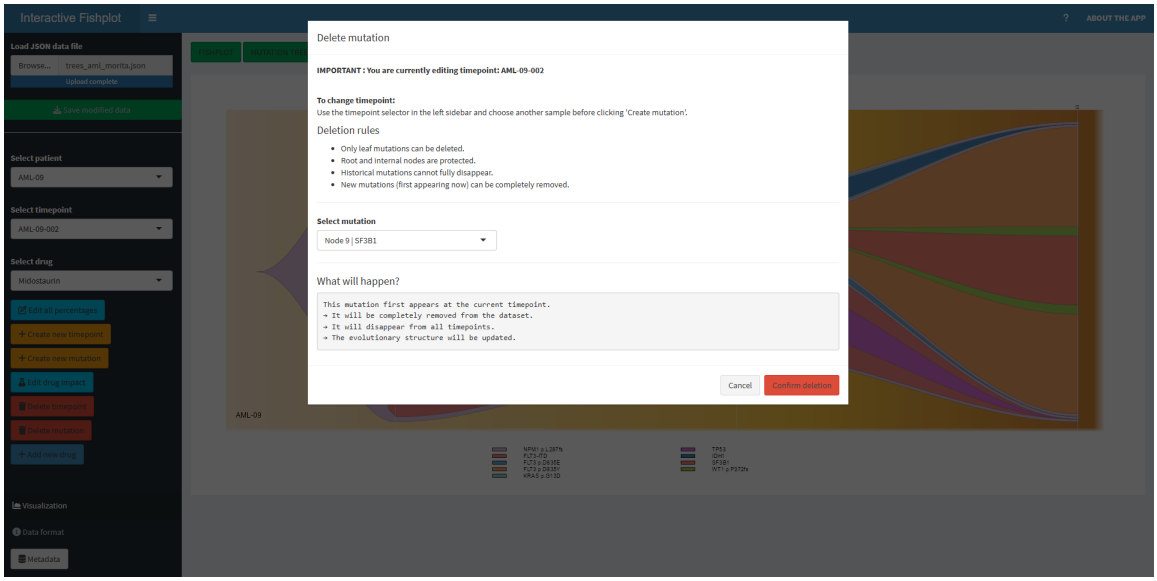


Figure 9: The *Delete Mutation* interface allows users to completly leaf mutations at a specific timepoint.The user is currently at timepoint 2. The deletion rules are displayed, and when a mutation is selected, a text is updated to indicate the expected outcome.

Venetoclax, *Cytarabine*, and *Azacitidine*. Each drug induces a specific effect on each mutation and for each sample.

3.5.1 Drug effects implementation

The drug effect is modeled as a multiplicative factor:

$$x_i\left(t+\tfrac{1}{2}\right) = x_i(t) \cdot d_i(t)$$

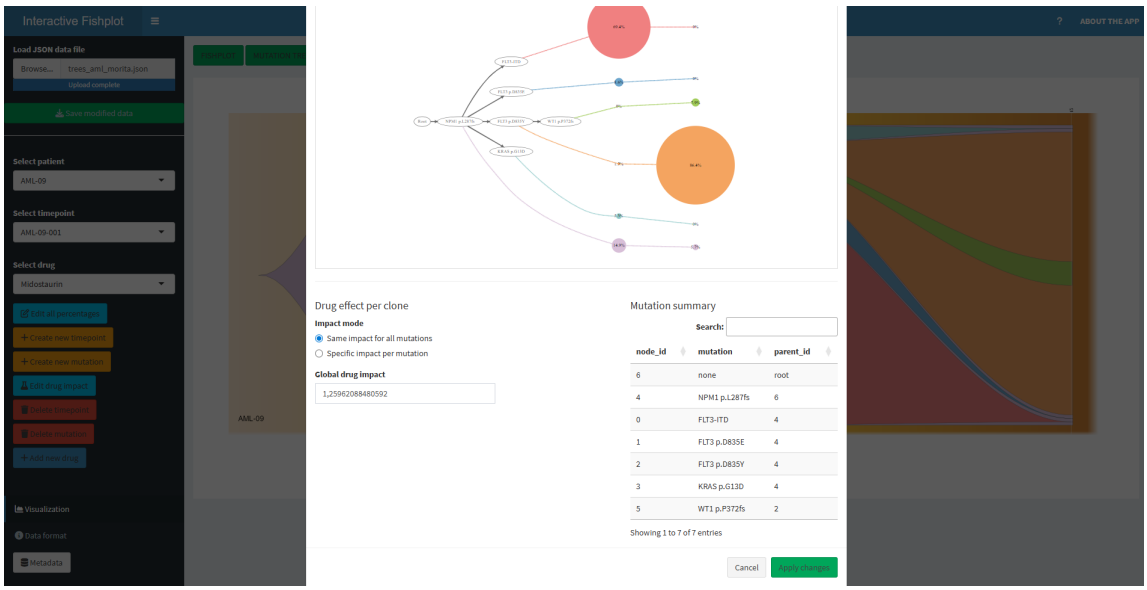


Figure 10: The global effect mode, where the drug effect is applied uniformly to all mutations.

Where:

- $i \in \{1, \ldots, N\}$ denotes a mutation,
- t denotes a real timepoint,
- $t + \frac{1}{2}$ denotes the intermediate (drug) timepoint,
- $x_i(t)$ is the abundance of mutation i at time t,
- $d_i(t)$ is the drug effect applied to mutation i at time t.

As a result, 3 different effects can be summarized as follows:

- $d_i(t) = 1$ corresponds to no effect,
- $0 \leq d_i(t) < 1$ induces a shrinkage of the clone,
- $d_i(t) > 1$ induces an expansion of the clone.

Two modes are available: the global mode, where a single scalar is applied uniformly to all mutations as shown in Figure 10, and the mutation-specific mode, which is defined independently for each mutation as shown in Figure 9.

Since mutations are structured as a clonal hierarchy, the abundances must satisfy consistency constraints. In particular, the abundance of a child clone cannot exceed that of its parent. As a result, after applying the drug effect, a rebalancing step is performed. Finally, drug effect values are constrained within $d_i(t) \in [0, 2]$, with a discretization step (e.g., 0.05) to allow fine control of the simulation.

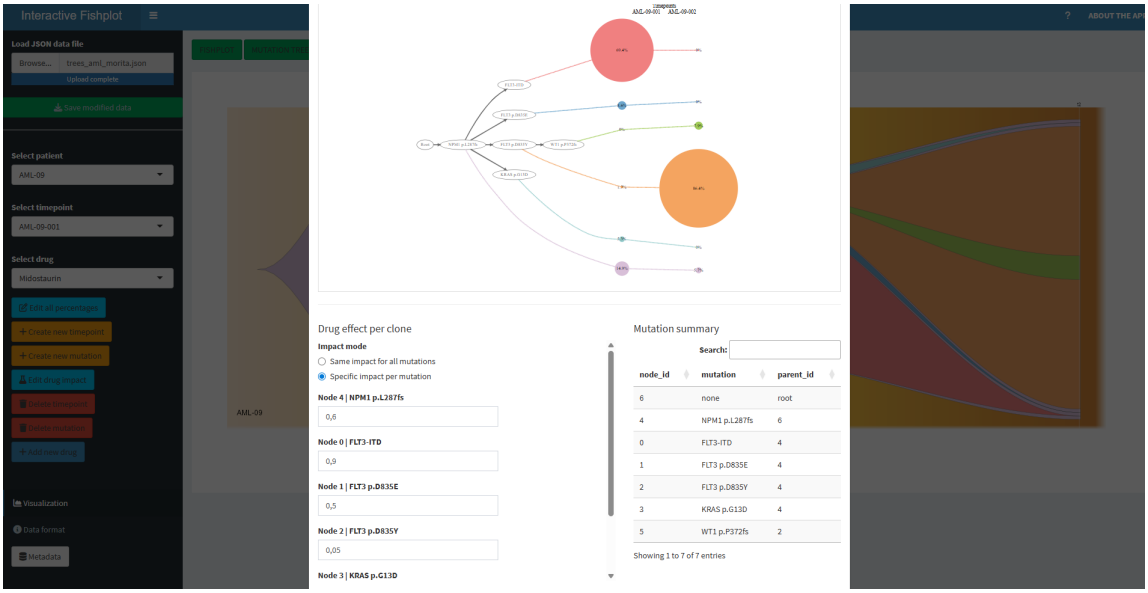


Figure 11: The mutation-specific effect mode where the drug effect is applied independently to each mutation

3.6 Robust handling of dynamic mutation sets

A specific technical issue arises when the set of mutations changes dynamically (e.g., when a new mutation such as TP53 is added to the clonal hierarchy). In this case, previously stored drug effect parameters may become inconsistent with the updated mutation set, leading to indexing errors during visualization. To address this, a synchronization step was introduced to ensure that the mutation-specific drug effect vector $d_i(t)$ remains aligned with the current set of mutations. Concretely, for each drug event, the set of mutation identifiers is compared to the names of the stored drug effect vector. Missing mutations are automatically initialized with a neutral effect ($d_i(t) = 1$), while obsolete entries are removed. This guarantees that the length and naming of the effect vector always match the current clonal structure. Importantly, this initialization does not restrict user interaction: newly added mutations appear in the interface with a default neutral effect, which can then be freely adjusted by the user in mutation-specific mode. This ensures both robustness of the application and full flexibility in defining drug responses. This correction ensures that the mutation-specific mode remains stable and functional even when the clonal architecture evolves over time, without requiring manual reinitialization through the global mode.

4. Discussion

An important contribution is the incorporation of biological constraints directly into the editing process. By enforcing rules such as parent-child consistency, total abundance limits, and root normalization, the framework ensures that user-defined modifications remain biologically plausible. This is particularly relevant in an interactive setting, where unconstrained manual edits could otherwise lead to unrealistic clonal configurations.

The introduction of drug effect simulation further enhances the exploratory capabilities of the tool. The multiplicative model provides a simple yet interpretable way to represent treatment impact on clonal populations. The availability of both global and mutation-specific modes allows users to simulate homogeneous or heterogeneous drug responses, which is consistent with known mechanisms of treatment resistance. However, this model remains a simplification of real biological processes, as it does not explicitly account for factors such as microenvironmental influences, stochastic effects, or complex pharmacodynamics.

Despite its strengths, several limitations should be acknowledged. First, the accuracy of the visualizations depends on the quality of the input data and the inferred mutation trees. Errors or uncertainties in phylogenetic reconstruction may propagate into the visualization and affect interpretation. Second, the drug effect model assumes independence between mutations, whereas in reality, interactions between clones may influence their dynamics. Third, the normalization and rebalancing steps, while necessary to enforce constraints, may introduce distortions that do not fully reflect true biological evolution.

In addition, the current framework focuses primarily on descriptive and exploratory analysis

rather than predictive modeling. Although users can simulate hypothetical scenarios, the system does not currently learn from data to predict future tumor evolution. Integrating statistical or machine learning models could further enhance its predictive capabilities.

Future work could extend this framework in several directions. Incorporating longitudinal clinical data and treatment histories would allow for more realistic simulations of therapeutic interventions. Improving the drug model by integrating dose-response relationships or evolutionary dynamics could also increase biological relevance. Finally, scaling the application to handle larger cohorts and enabling comparative analyses across patients would provide additional insights into conserved evolutionary patterns.

4.1 Conclusion

In conclusion, this application offers a novel and intuitive approach to exploring tumor evolution, combining visualization, interactivity, and biological constraints. By integrating user-driven editing and drug effect simulation, the application provides a flexible environment, while also highlighting opportunities for further methodological developments.

A major strength of this approach lies in its ability to bridge the gap between phylogenetic reconstruction and temporal visualization. While existing methods such as SCITE or COMPASS accurately infer mutation trees, they often lack intuitive tools to explore how clonal populations evolve over time under different conditions. The proposed application addresses this limitation by coupling tree structures with dynamic abundance profiles, enabling users to better interpret tumor heterogeneity and its progression.

Acknowledgements

I would like to sincerely thank my supervisor, Luo Xiang Ge, for her invaluable advice, guidance, and the time she dedicated to me throughout this internship. I would also like to thank all the members of Niko Beerenwinkel's lab for their valuable feedback on my work. Finally, I would like to express my gratitude to Titouan Lestanguet for his help.

References

[1] Ortmann CA, Kent DG, Nangalia J, Silber Y, Wedge DC, Grinfeld J, Baxter EJ, Massie CE, Papaemmanuil E, Menon S, Godfrey AL, Dimitropoulou D, Guglielmelli P, Bellosillo B, Besses C, Döhner K, Harrison CN, Vassiliou GS, Vannucchi A, Campbell PJ, Green AR. Effect of mutation order on myeloproliferative neoplasms. *N Engl J Med.* 2015 Feb 12;372(7):601-612, doi: 10.1056/NEJMoa1412098

[2] Jahn, K., Kuipers, J. & Beerenwinkel, N. Tree inference for single-cell data. *Genome Biol*, **17**, 86 (2016). https://doi.org/10.1186/s13059-016-0936-x

[3] Sollier, E., Kuipers, J., Takahashi, K. et al. COMPASS: joint copy number and mutation phylogeny reconstruction from amplicon single-cell sequencing data. *Nat Commun* **14**, 4921 (2023). https://doi.org/10.1038/s41467-023-40378-8

[4] Luo, Xiang Ge and Kuipers, Jack and Rupp, Kevin and Takahashi, Koichi and Beerenwinkel, Niko, Bayesian inference of fitness landscapes via tree-structured branching processes, *bioRxiv*, 2025 https://www.biorxiv.org/content/early/2025/03/27/2025.01.24.634649

[5] Luo, X.G., Kuipers, J. & Beerenwinkel, N. Joint inference of exclusivity patterns and recurrent trajectories from tumor mutation trees. *Nat Commun* **14**, 3676 2023 https://doi.org/10.1038/s41467-023-39400-w

[6] Monica-Andreea Baciu-Drăgan, Niko Beerenwinkel, Oncotree2vec — a method for embedding and clustering of tumor mutation trees, *Bioinformatics*Volume 40, July 2024, Pages i180–i188, https://doi.org/10.1093/bioinformatics/btae214

[7] Burrell, R., McGranahan, N., Bartek, J. et al. The causes and consequences of genetic heterogeneity in cancer evolution, *Nature* 501, 338–345 (2013). https://doi.org/10.1038/nature12625

[8] Ding, L., Ley, T., Larson, D. et al. Clonal evolution in relapsed acute myeloid leukaemia revealed by whole-genome sequencing, *Nature* 481, 506–510 (2012). https://doi.org/10.1038/nature10738

[9] Marusyk, A., Almendro, V. & Polyak, K. Intra-tumour heterogeneity: a looking glass for cancer? *Nat Rev Cancer* 12, 323–334 (2012). https://doi.org/10.1038/nrc3261

[10] Nicholas McGranahan, Charles Swanton, Biological and Therapeutic Impact of Intratumor Heterogeneity in Cancer Evolution, *Cancer Cell* Volume 27, Issue 1,2015, Pages 15-26, ISSN 1535-6108, https://doi.org/10.1016/j.ccell.2014.12.001

[11] Jack Kuipers, Katharina Jahn, Niko Beerenwinkel, Advances in understanding tumour evolution through single-cell sequencing, *Biochimica et Biophysica Acta (BBA) - Reviews on Cancer,* Volume 1867, Issue 2, 2017, Pages 127-138, ISSN 0304-419X, https://doi.org/10.1016/j.bbcan.2017.02.001

[12] Monica-Andreea Baciu-Drăgan, Niko Beerenwinkel, OncotreeVIS – an interactive graphical user interface for visualizing mutation tree cohorts, *bioRxiv* 2024.11.15.623847; https://doi.org/10.1101/2024.11.15.623847

[13] Miller, C.A., McMichael, J., Dang, H.X. et al. Visualizing tumor evolution with the fishplot package for R. *BMC Genomics* 17, 880 (2016). https://doi.org/10.1186/s12864-016-3195-z

[14] Morita, K., Wang, F., Jahn, K. et al. Clonal evolution of acute myeloid leukemia revealed by high-throughput single-cell genomics. *Nat* Commun 11, 5327 (2020). https://doi.org/10.1038/s41467-020-19119-8